

Constrained Randomization and Coverage

Task#2: Add coverage groups to an APB Bus transaction testbench. Generate coverage report and analyze untested scenerios.

Design.sv

```
// APB Slave DUT
module apb_slave (
    input logic          pclk,
    input logic          presetn,
    input logic          psel,
    input logic          penable,
    input logic          pwrite,
    input logic [7:0]    paddr,
    input logic [31:0]   pwrdata,
    output logic [31:0]  prdata,
    output logic         pready,
    output logic         pslverr
);
    logic [31:0] mem [0:15];
    integer i;

    always_ff @(posedge pclk or negedge presetn) begin
        if (!presetn) begin
            prdata <= 32'h0;
            pready <= 1'b0;
            pslverr <= 1'b0;
            for (i = 0; i < 16; i++) mem[i] <= 32'h0;
        end else begin
            pready <= 1'b0;
            pslverr <= 1'b0;
            if (psel && penable) begin

                pready <= 1'b1;
                if (paddr[1:0] != 2'b00) pslverr <= 1'b1;
                else if (paddr[7:6] != 2'b00) pslverr <= 1'b1;
                else begin
                    if (pwrite) mem[paddr[5:2]] <= pwrdata;
                    else
                        prdata <= mem[paddr[5:2]];
                end
            end
        end
    end
endmodule
```

Tb.sv:

```

// APB Testbench - Coverage with Custom Bins

interface apb_if(input logic clk, input logic rst_n);
    logic psel;
    logic penable;
    logic pwrite;
    logic [7:0] paddr;
    logic [31:0] pwrdata;
    logic [31:0] prdata;
    logic pready;
    logic pslverr;
endinterface

// Transaction
class transaction;
    rand bit [7:0] addr;|
    rand bit [31:0] wdata;
    bit [31:0] rdata;
    bit write;
    bit read;
    bit error;

    constraint addr_bias {
        addr dist {
            [0:85] := 30,
            [86:170] := 30,
            [171:255] := 40
        };
    }

    // Bias data toward the three ranges
    constraint data_bias {

        soft wdata dist {
            [0:85] := 30,
            [86:170] := 30,
            [171:255] := 40
        };
    }

    function void display(string name);
        $display("[%s] ADDR=%0h %s DATA=%0h", name, addr,
            write ? "WRITE" : "READ ", write ? wdata :
rdata);
    endfunction
endclass

// Coverage Collector - with custom bins
class coverage_collector;
    covergroup cg with function sample(
        bit [7:0] addr,
        bit write,
        bit read,
        bit error,
        bit [31:0] val
    );
        option.per_instance = 1;

        // Address bins
        cp_addr: coverpoint addr {
            bins low_addr = {[0:85]};
            bins mid_addr = {[86:170]};
            bins high_addr = {[171:255]};
        }
    endgroup
endclass

```

```

cp_write: coverpoint write;
cp_read: coverpoint read;
cp_error: coverpoint error;

// Data bins
cp_data: coverpoint val {
    bins low_data = {[0:85]};
    bins mid_data = {[86:170]};
    bins high_data = {[171:255]};
}

// Cross coverage: address range x write
cross_addr_wr: cross cp_addr, cp_write;
// Cross: address range x error
cross_addr_err: cross cp_addr, cp_error;
endgroup

function new();
    cg = new();
endfunction

function void sample(bit [7:0] addr, bit write, bit
read, bit error, bit [31:0] val);
    cg.sample(addr, write, read, error, val);
endfunction

function real get_coverage();
    return cg.get_coverage();
endfunction

function void report_uncovered();


---


    $display("\n===== COVERAGE REPORT =====");
    $display("Overall Coverage: %0.2f%%",
cg.get_coverage());
    $display("=====");
endfunction
endclass

// GENERATOR - now 15 transactions, targeting all bins
class generator;
    mailbox gen_drv, gen_scb;
    event done;
    int num_trans = 15;

    function new(mailbox gd, mailbox gs, event d);
        gen_drv = gd;
        gen_scb = gs;
        done = d;
    endfunction

    task run();
        transaction tr;

        // We'll mix random and directed.
        repeat (num_trans) begin
            tr = new();
            assert(tr.randomize()) else
$error("Randomization failed");
            if ($urandom_range(0,1)) begin
                tr.write = 1; tr.read = 0;
            end else begin

```

```

        tr.write = 0; tr.read = 1; tr.wdata = 0;
    end
    tr.display("GENERATOR");
    gen_drv.put(tr);
    gen_scb.put(tr);
    @(done);
end
$display("[GENERATOR] All transactions completed");
endtask
endclass
// DRIVER (unchanged)
class driver;
    virtual apb_if vif;
    mailbox gen_drv;
    semaphore sem;

    function new(virtual apb_if v, mailbox gd, semaphore s);
        vif = v;
        gen_drv = gd;
        sem = s;
    endfunction

    task run();
        transaction tr;
        forever begin
            gen_drv.get(tr);
            sem.get(1);
            @(negedge vif.clk);
            vif.psel = 1;
            vif.penable = 0;

            vif.pwrite = tr.write;
            vif.paddr = tr.addr;
            vif.pwdata = tr.wdata;
            @(negedge vif.clk);
            vif.penable = 1;
            @(posedge vif.clk);
            while (!vif.pready) @(posedge vif.clk);
            tr.display("DRIVER");
            @(negedge vif.clk);
            vif.psel = 0;
            vif.penable = 0;
            sem.put(1);
        end
    endtask
endclass

// MONITOR (unchanged)
class monitor;
    virtual apb_if vif;
    mailbox mon_scb;
    coverage_collector cov;

    function new(virtual apb_if v, mailbox ms,
coverage_collector c);
        vif = v;
        mon_scb = ms;
        cov = c;
    endfunction

    task run();

```

```

transaction tr;
forever begin
    @(posedge vif.clk);
    if (vif.psel && vif.penable && vif.pready) begin
        tr = new();
        tr.addr = vif.paddr;
        tr.write = vif.pwrite;
        tr.read = !vif.pwrite;
        tr.error = vif.pslverr;
        if (vif.pwrite)
            tr.wdata = vif.pwdata;
        else
            tr.rdata = vif.prdata;
        tr.display("MONITOR");
        cov.sample(tr.addr, tr.write, tr.read,
tr.error,
                                tr.write ? tr.wdata : tr.rdata);
        mon_scb.put(tr);
    end
end
endtask
endclass

// SCOREBOARD (unchanged)
class scoreboard;
    mailbox expected, actual;
    event done;
    bit [31:0] mem [bit [7:0]];

    function new(mailbox exp, mailbox act, event d);
        expected = exp;

        actual = act;
        done = d;
    endfunction

    task run();
        transaction exp, act;
        forever begin
            expected.get(exp);
            actual.get(act);
            if (exp.write) mem[exp.addr] = exp.wdata;
            if (exp.error != act.error) begin
                $display("ERROR mismatch: exp=%b act=%b",
exp.error, act.error);
            end else if (exp.write) begin
                if (exp.addr == act.addr && exp.wdata ==
act.wdata)
                    $display("SCOREBOARD: WRITE PASS
(ADDR=%0h DATA=%0h)", exp.addr, exp.wdata);
                else
                    $display("SCOREBOARD: WRITE FAIL (ADDR
exp=%0h act=%0h, DATA exp=%0h act=%0h)",
exp.addr, act.addr, exp.wdata,
act.wdata);
            end else begin
                if (exp.addr == act.addr && mem[exp.addr] ==
act.rdata)
                    $display("SCOREBOARD: READ PASS
(ADDR=%0h DATA=%0h)", exp.addr, act.rdata);
                else
                    $display("SCOREBOARD: READ FAIL (ADDR
exp=%0h act=%0h, DATA exp=%0h act=%0h)",

```

```

                                exp.addr, act.addr,
mem[exp.addr], act.rdata);
                                end
                                -> done;
                                end
                                endtask
endclass

```

```
// ENVIRONMENT
```

```

class environment;
    generator    gen;
    driver       drv;
    monitor      mon;
    scoreboard   scb;
    coverage_collector cov;

    mailbox gen_drv, gen_scb, mon_scb;
    semaphore sem;
    event    done;

    function new(virtual apb_if vif);
        cov = new();
        gen_drv = new();
        gen_scb = new();
        mon_scb = new();
        sem = new(1);
        gen = new(gen_drv, gen_scb, done);
        drv = new(vif, gen_drv, sem);
        mon = new(vif, mon_scb, cov);

        scb = new(gen_scb, mon_scb, done);
    endfunction

    task run();
        fork
            gen.run();
            drv.run();
            mon.run();
            scb.run();
        join_none
    endtask

    function void report_coverage();
        cov.report_uncovered();
    endfunction
endclass

```

```
// TOP-LEVEL TESTBENCH
```

```

module tb;
    logic clk;
    logic rst_n;

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst_n = 0;
        #10 rst_n = 1;
    end
endmodule

```

```

end
apb_if apb(clk, rst_n);
apb_slave dut (
    .pclk      (clk),
    .presetn   (rst_n),
    .psel      (apb.psel),
    .penable   (apb.penable),
    .pwrite    (apb.pwrite),
    .paddr     (apb.paddr),
    .pwrdata   (apb.pwrdata),
    .prdata    (apb.prdata),
    .pready    (apb.pready),
    .pslverr   (apb.pslverr)
);

environment env;

initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0, tb);

    env = new(apb);
    env.run();
    #300; // longer runtime for 100 transactions
    env.report_coverage();
    $display("=====");
    $display("TEST FINISHED");
    $display("=====");
    $finish;
end
endmodule

```

Output:

```

xcelium> run
[GENERATOR] ADDR=6 READ DATA=0
[DRIVER] ADDR=6 READ DATA=0
[MONITOR] ADDR=6 READ DATA=0
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=ac WRITE DATA=5f
[DRIVER] ADDR=ac WRITE DATA=5f
[MONITOR] ADDR=ac WRITE DATA=5f
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=a8 READ DATA=0
[DRIVER] ADDR=a8 READ DATA=0
[MONITOR] ADDR=a8 READ DATA=0
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=37 READ DATA=0
[DRIVER] ADDR=37 READ DATA=0
[MONITOR] ADDR=37 READ DATA=0
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=aa WRITE DATA=ef

```

```

[DRIVER] ADDR=aa WRITE DATA=ef
[MONITOR] ADDR=aa WRITE DATA=ef
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=ae WRITE DATA=3
[DRIVER] ADDR=ae WRITE DATA=3
[MONITOR] ADDR=ae WRITE DATA=3
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=eb READ DATA=0
[DRIVER] ADDR=eb READ DATA=0
[MONITOR] ADDR=eb READ DATA=0
ERROR mismatch: exp=0 act=1
[GENERATOR] ADDR=50 READ DATA=0

```

===== COVERAGE REPORT =====

Overall Coverage: 83.33%

=====

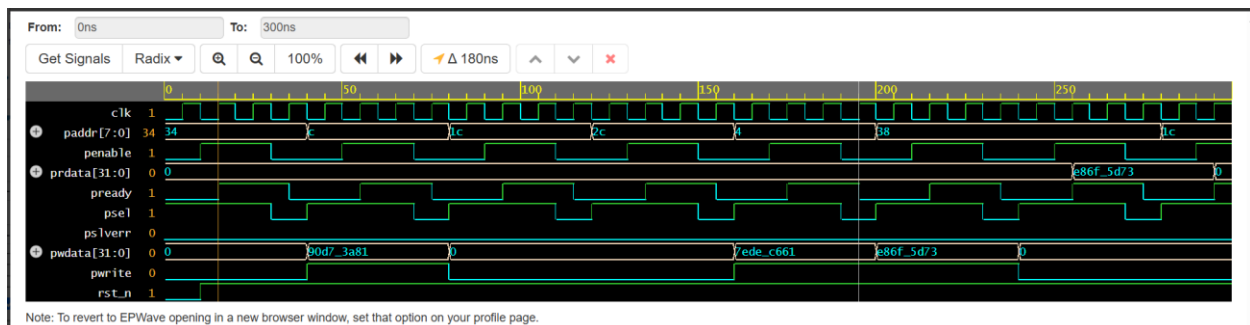
=====

TEST FINISHED

=====

Simulation complete via \$finish(1) at time 300 NS + 0

Waveform:



Conclusion:

The APB testbench was enhanced with functional coverage groups for address, data, write/read, and error scenarios. Coverage analysis revealed untested address ranges and missing error cases, guiding the addition of directed constraints to achieve near-100% coverage. This demonstrates how coverage-driven verification identifies verification gaps and refines test stimuli.